

UNIVERSIDAD PRIVADA DOMINGO SAVIO
Diseño Web II

ENTREGA 2B

Login, Registro y Redirect por Rol

Proyecto: gestión-académica

Fase 1 — Fundación | Prerrequisito: Entregas 1A, 1B y 2A completadas

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear un layout específico para páginas de autenticación (centrado, sin sidebar).
- Construir formularios de login y registro funcionales con shadcn/ui y validación Zod.
- Implementar una API Route que registre al usuario en Supabase Auth Y cree su perfil en la BD.
- Configurar la redirección automática según el rol del usuario después del login.
- Comprender la diferencia entre componentes "use client" y Server Components.

Prerrequisitos

- ✓ Entregas 1A, 1B y 2A completadas
- ✓ Archivos `lib/supabase/client.ts` y `server.ts` creados
- ✓ `proxy.ts` configurado en la raíz del proyecto
- ✓ `Callback auth/callback/route.ts` implementado

Teoría: "use client" vs Server Components

Antes de construir los formularios, necesitas entender una distinción fundamental en Next.js 16:

	Server Components	Client Components
Se ejecutan en	Solo en el servidor	Servidor (1era carga) + navegador
Directiva	Ninguna (es el default)	"use client" al inicio del archivo
Pueden usar	await, acceso a BD directo, cookies	useState, useEffect, onClick, onChange
NO pueden usar	useState, eventos del navegador	Acceso directo a BD, cookies()
Ejemplo	Páginas que muestran datos	Formularios, botones, modales

Regla simple: si tu componente necesita interactividad (formularios, clics, estados), necesita **"use client"**. Si solo muestra datos, déjalo como Server Component.

Crear el layout de autenticación

El layout de autenticación envolverá las páginas de login y registro. Su diseño es simple: centra el contenido vertical y horizontalmente en la pantalla, sin sidebar ni header.

Crea el archivo **app/(auth)/layout.tsx**:

```
Archivo: app/(auth)/layout.tsx
export default function AuthLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <div className="flex min-h-screen items-center justify-center bg-muted/40">
      <div className="w-full max-w-md px-4">
        {children}
      </div>
    </div>
  )
}
```

Este layout es un **Server Component** (no tiene "use client"). Solo define la estructura visual. No necesita interactividad.

La clase `bg-muted/40` usa una variable CSS de shadcn que produce un fondo gris suave. El `max-w-md` limita el ancho a 448px, ideal para formularios.

Crear el componente de formulario de login

El formulario de login es un Client Component porque necesita manejar estados (email, password, loading, errores) y eventos (onSubmit).

Crea el archivo **components/auth/login-form.tsx**:

```
Archivo: components/auth/login-form.tsx
"use client"

import { useState } from "react"
import { useRouter } from "next/navigation"
import Link from "next/link"
import { createClient } from "@lib/supabase/client"

import { Button } from "@components/ui/button"
import { Input } from "@components/ui/input"
import { Label } from "@components/ui/label"
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
}
```

```

    CardTitle,
  } from "@components/ui/card"

export function LoginForm() {
  const router = useRouter()
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")
  const [error, setError] = useState<string | null>(null)
  const [loading, setLoading] = useState(false)

  async function handleLogin(e: React.FormEvent) {
    e.preventDefault()
    setError(null)
    setLoading(true)

    try {
      const supabase = createClient()

      // 1. Autenticar con Supabase
      const { error: authError } = await supabase.auth.signInWithPassword({
        email,
        password,
      })

      if (authError) {
        setError(authError.message)
        return
      }

      // 2. Obtener el perfil del usuario para saber su rol
      const { data: { user } } = await supabase.auth.getUser()

      if (!user) {
        setError("No se pudo obtener el usuario")
        return
      }

      const { data: perfil } = await supabase
        .from("usuarios_perfil")
        .select("rol")
        .eq("id", user.id)
        .single()

      // 3. Redirigir según el rol
      if (perfil) {
        switch (perfil.rol) {
          case "estudiante":
            router.push("/estudiante")
            break
          case "docente":
            router.push("/docente")

```

```

        break
      case "admin":
        router.push("/admin")
        break
      default:
        router.push("/")
    }
  } else {
    router.push("/")
  }

  // 4. Forzar refresh para que proxy.ts actualice cookies
  router.refresh()
} catch {
  setError("Error inesperado. Intenta de nuevo.")
} finally {
  setLoading(false)
}
}

return (
  <Card>
    <CardHeader className="text-center">
      <CardTitle className="text-2xl">Gestión Académica</CardTitle>
      <CardDescription>
        Ingresa tus credenciales para acceder al sistema
      </CardDescription>
    </CardHeader>
    <form onSubmit={handleLogin}>
      <CardContent className="space-y-4">
        {error && (
          <div className="rounded-md bg-destructive/10 p-3 text-sm text-destructive">
            {error}
          </div>
        )}
        <div className="space-y-2">
          <Label htmlFor="email">Correo electrónico</Label>
          <Input
            id="email"
            type="email"
            placeholder="tu@email.com"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            disabled={loading}
          />
        </div>
        <div className="space-y-2">
          <Label htmlFor="password">Contraseña</Label>
          <Input
            id="password"
            type="password"

```

```

        placeholder="••••••••"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        required
        minLength={6}
        disabled={loading}
      />
    </div>
  </CardContent>
  <CardFooter className="flex flex-col gap-3">
    <Button type="submit" className="w-full" disabled={loading}>
      {loading ? "Ingresando..." : "Iniciar Sesión"}
    </Button>
    <p className="text-sm text-muted-foreground">
      ¿No tienes cuenta?{" " }
      <Link href="/registro" className="text-primary hover:underline">
        Regístrate aquí
      </Link>
    </p>
  </CardFooter>
</form>
</Card>
)
}

```

Desglose del componente

Línea 1: "use client" — obligatorio porque usamos useState, useRouter y eventos de formulario.

Líneas 22-25: Cuatro estados para controlar el formulario: email, password, error (mensaje de error), loading (deshabilitar botón mientras procesa).

Línea 35: signInWithPassword() es el método de Supabase para autenticar con email y contraseña. Retorna un error si las credenciales son inválidas.

Líneas 49-54: Después del login, consultamos la tabla usuarios_perfil para obtener el rol. Usamos .single() porque esperamos exactamente un resultado.

Línea 77: router.refresh() fuerza a Next.js a re-ejecutar proxy.ts, lo que actualiza las cookies de sesión en el servidor.

Nota: ¿Por qué no usamos react-hook-form aquí?

El formulario de login es simple (2 campos). Usamos useState directo para mantener la complejidad baja. En formularios más complejos (registro, editar perfil, crear materia) sí usaremos react-hook-form + Zod.

Crear la página de login

La página es un Server Component ultra-simple que solo renderiza el componente LoginForm. Crea `app/(auth)/login/page.tsx`:

```
Archivo: app/(auth)/login/page.tsx
import { LoginForm } from "@/components/auth/login-form"

export default function LoginPage() {
  return <LoginForm />
}
```

¿Por qué separar el componente de la página? El archivo `page.tsx` debe ser Server Component por defecto (para SEO y rendimiento). La interactividad vive en `LoginForm` que es un Client Component importado. Este patrón se llama **composición server/client** y lo usarás constantemente.

Crear el componente de formulario de registro

El formulario de registro es más complejo que el de login. Incluye campos adicionales (nombre, rol, carrera) y realiza una doble operación: crear usuario en Supabase Auth y crear su perfil en la BD a través de una API Route.

Crea el archivo `components/auth/registro-form.tsx`:

```
Archivo: components/auth/registro-form.tsx
"use client"

import { useState } from "react"
import Link from "next/link"

import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from "@/components/ui/card"

export function RegistroForm() {
  const [formData, setFormData] = useState({
    email: "",
    password: "",
    nombre_completo: "",
    rol: "estudiante",
    carrera: "",
  })
}
```

```

})
const [error, setError] = useState<string | null>(null)
const [success, setSuccess] = useState(false)
const [loading, setLoading] = useState(false)

function handleChange(
  e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>
) {
  setFormData((prev) => ({
    ...prev,
    [e.target.name]: e.target.value,
  }))
}

async function handleRegistro(e: React.FormEvent) {
  e.preventDefault()
  setError(null)
  setLoading(true)

  try {
    // Llamar a nuestra API Route para crear usuario + perfil
    const response = await fetch("/api/auth/registro", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData),
    })

    const data = await response.json()

    if (!response.ok) {
      setError(data.error || "Error al registrar")
      return
    }

    // Registro exitoso
    setSuccess(true)
  } catch {
    setError("Error de conexión. Intenta de nuevo.")
  } finally {
    setLoading(false)
  }
}

// Si el registro fue exitoso, mostrar mensaje de verificación
if (success) {
  return (
    <Card>
      <CardHeader className="text-center">
        <CardTitle className="text-2xl text-green-600">
          ¡Registro Exitoso!
        </CardTitle>
      </CardHeader>
    </Card>
  )
}

```



```

        <CardDescription>
            Hemos enviado un enlace de verificación a tu correo
            electrónico. Por favor, revisa tu bandeja de entrada
            (y la carpeta de spam) para confirmar tu cuenta.
        </CardDescription>
    </CardHeader>
    <CardFooter className="justify-center">
        <Link href="/login">
            <Button variant="outline">Ir a Iniciar Sesión</Button>
        </Link>
    </CardFooter>
</Card>
)
}

return (
    <Card>
        <CardHeader className="text-center">
            <CardTitle className="text-2xl">Crear Cuenta</CardTitle>
            <CardDescription>
                Completa tus datos para registrarte en el sistema
            </CardDescription>
        </CardHeader>
        <form onSubmit={handleRegistro}>
            <CardContent className="space-y-4">
                {error && (
                    <div className="rounded-md bg-destructive/10 p-3 text-sm text-
destructive">
                        {error}
                    </div>
                )}

                <div className="space-y-2">
                    <Label htmlFor="nombre_completo">Nombre completo</Label>
                    <Input
                        id="nombre_completo"
                        name="nombre_completo"
                        placeholder="Juan Pérez Mamani"
                        value={formData.nombre_completo}
                        onChange={handleChange}
                        required
                        disabled={loading}
                    />
                </div>

                <div className="space-y-2">
                    <Label htmlFor="email">Correo electrónico</Label>
                    <Input
                        id="email"
                        name="email"
                        type="email"
                        placeholder="tu@email.com"

```

```

        value={formData.email}
        onChange={handleChange}
        required
        disabled={loading}
      />
    </div>

    <div className="space-y-2">
      <Label htmlFor="password">Contraseña</Label>
      <Input
        id="password"
        name="password"
        type="password"
        placeholder="Mínimo 6 caracteres"
        value={formData.password}
        onChange={handleChange}
        required
        minLength={6}
        disabled={loading}
      />
    </div>

    <div className="space-y-2">
      <Label htmlFor="rol">Rol</Label>
      <select
        id="rol"
        name="rol"
        value={formData.rol}
        onChange={handleChange}
        disabled={loading}
        className="flex h-10 w-full rounded-md border border-input bg-
background px-3 py-2 text-sm ring-offset-background focus-visible:outline-none focus-
visible:ring-2 focus-visible:ring-ring"
      >
        <option value="estudiante">Estudiante</option>
        <option value="docente">Docente</option>
        <option value="admin">Administrador</option>
      </select>
    </div>

    <div className="space-y-2">
      <Label htmlFor="carrera">Carrera (opcional)</Label>
      <Input
        id="carrera"
        name="carrera"
        placeholder="Ej: Ingeniería de Sistemas"
        value={formData.carrera}
        onChange={handleChange}
        disabled={loading}
      />
    </div>
  </CardContent>

```

```

    <CardFooter className="flex flex-col gap-3">
      <Button type="submit" className="w-full" disabled={loading}>
        {loading ? "Registrando..." : "Crear Cuenta"}
      </Button>
      <p className="text-sm text-muted-foreground">
        ¿Ya tienes cuenta?{" "}
        <Link href="/login" className="text-primary hover:underline">
          Inicia sesión
        </Link>
      </p>
    </CardFooter>
  </form>
</Card>
)
}

```

Aspectos clave del formulario de registro

Estado agrupado: En lugar de un useState por campo, usamos un objeto formData con todos los campos. La función handleChange actualiza cualquier campo dinámicamente usando [e.target.name] como clave.

API Route: El formulario NO llama a Supabase directamente. En su lugar, hace un fetch a /api/auth/registro. Esto es intencional porque necesitamos crear el perfil con la Service Role Key (que solo está en el servidor).

Estado de éxito: Si el registro es exitoso, el formulario completo se reemplaza por un mensaje de verificación. Esto evita que el usuario intente registrarse dos veces.

Select nativo: Usamos un <select> HTML con clases de Tailwind que imitan el estilo de shadcn. En entregas futuras lo reemplazaremos por el componente Select de shadcn.

Crear la página de registro

Crea el archivo `app/(auth)/registro/page.tsx`:

```

Archivo: app/(auth)/registro/page.tsx
import { RegistroForm } from "@/components/auth/registro-form"

export default function RegistroPage() {
  return <RegistroForm />
}

```

Crear la API Route de registro

Esta es la API Route más importante de la aplicación. Realiza dos operaciones atómicas: crear el usuario en Supabase Auth y crear su perfil en la tabla usuarios_perfil.

Crea el archivo **app/api/auth/registro/route.ts**:

```

Archivo: app/api/auth/registro/route.ts
import { createClient } from "@supabase/supabase-js"
import { NextResponse } from "next/server"

// Cliente admin con Service Role Key (bypasea RLS)
const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

export async function POST(request: Request) {
  try {
    const body = await request.json()
    const { email, password, nombre_completo, rol, carrera } = body

    // — Validación básica —
    if (!email || !password || !nombre_completo || !rol) {
      return NextResponse.json(
        { error: "Todos los campos obligatorios deben completarse" },
        { status: 400 }
      )
    }

    if (password.length < 6) {
      return NextResponse.json(
        { error: "La contraseña debe tener al menos 6 caracteres" },
        { status: 400 }
      )
    }

    const rolesValidos = ["estudiante", "docente", "admin"]
    if (!rolesValidos.includes(rol)) {
      return NextResponse.json(
        { error: "Rol no válido" },
        { status: 400 }
      )
    }

    // — 1. Crear usuario en Supabase Auth —
    const { data: authData, error: authError } =
      await supabaseAdmin.auth.admin.createUser({
        email,
        password,
        email_confirm: false, // El usuario debe confirmar por email
      })
  }
}

```

```

    if (authError) {
      return NextResponse.json(
        { error: authError.message },
        { status: 400 }
      )
    }

    // — 2. Crear perfil en la tabla usuarios_perfil —
    const { error: profileError } = await supabaseAdmin
      .from("usuarios_perfil")
      .insert({
        id: authData.user.id, // Mismo UUID que auth.users
        nombre_completo,
        rol,
        carrera: carrera || null,
      })

    if (profileError) {
      // Si falla el perfil, eliminar el usuario de Auth para no dejar datos
      // huérfanos
      await supabaseAdmin.auth.admin.deleteUser(authData.user.id)

      return NextResponse.json(
        { error: "Error al crear el perfil: " + profileError.message },
        { status: 500 }
      )
    }

    // — 3. Respuesta exitosa —
    return NextResponse.json(
      {
        message: "Usuario registrado exitosamente",
        user: { id: authData.user.id, email },
      },
      { status: 201 }
    )
  } catch {
    return NextResponse.json(
      { error: "Error interno del servidor" },
      { status: 500 }
    )
  }
}

```

Desglose de la API Route

Línea 5-8 — Cliente Admin: Usamos `createClient` directamente (NO el de `lib/supabase`) con la `SUPABASE_SERVICE_ROLE_KEY`. Esta clave tiene acceso total y `bypasea` RLS. SOLO se usa en el servidor.

Líneas 16-37 — Validación: Verificamos campos obligatorios, largo de contraseña y roles válidos ANTES de tocar la BD. Esto evita errores de constraint a nivel de PostgreSQL.

Línea 41 — `admin.createUser`: Usamos el método `admin` en lugar del método público para tener control total. Con `email_confirm: false`, Supabase envía el email de confirmación automáticamente.

Línea 56 — Mismo UUID: El campo `id` del perfil usa el MISMO UUID que `auth.users`. Esto mantiene la relación 1:1 entre la tabla de autenticación y la de perfiles.

Líneas 64-69 — Rollback manual: Si crear el perfil falla, eliminamos al usuario de Auth. Esto es un rollback manual para evitar usuarios sin perfil (datos huérfanos).

Seguridad: SUPABASE_SERVICE_ROLE_KEY

Esta clave NUNCA debe llegar al navegador. Verifica que la variable NO tiene el prefijo `NEXT_PUBLIC_`. Si alguien obtiene esta clave, puede leer, escribir y eliminar cualquier dato de tu base de datos sin restricciones.

Actualizar `proxy.ts` con redirección por rol

Ahora que tenemos el login funcionando, vamos a mejorar `proxy.ts` para que redirija a cada usuario a su dashboard correcto según su rol.

Abre `proxy.ts` y reemplaza la sección de redirección (líneas después de `auth.getUser`):

Reemplazo parcial en: `proxy.ts`

```
// ... (mantener todo hasta después de auth.getUser())

// 4. Definir rutas públicas
const publicRoutes = ["/login", "/registro", "/auth"]
const isPublicRoute = publicRoutes.some((route) =>
  request.nextUrl.pathname.startsWith(route)
)

// 5. Si no hay usuario y la ruta NO es pública
if (!user && !isPublicRoute && request.nextUrl.pathname !== "/") {
  const url = request.nextUrl.clone()
  url.pathname = "/login"
  return NextResponse.redirect(url)
}

// 6. Si hay usuario y está en ruta de auth, redirigir a su dashboard
if (user && isPublicRoute) {
  // Consultar el rol del usuario
  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single()
```

```

const url = request.nextUrl.clone()

if (perfil) {
  switch (perfil.rol) {
    case "estudiante":
      url.pathname = "/estudiante"
      break
    case "docente":
      url.pathname = "/docente"
      break
    case "admin":
      url.pathname = "/admin"
      break
    default:
      url.pathname = "/"
  }
} else {
  url.pathname = "/"
}

return NextResponse.redirect(url)
}

return supabaseResponse

```

Nota sobre rendimiento

Esta consulta a `usuarios_perfil` se ejecuta en cada request a rutas públicas cuando hay usuario autenticado. Para un MVP esto es aceptable. En producción, se optimiza guardando el rol en los claims del JWT de Supabase.

Probar el flujo completo

Es momento de probar todo el flujo de autenticación de principio a fin:

1. **Inicia el servidor:** `npm run dev`
2. Ve a **`http://localhost:3000/registro`**
3. Completa el formulario con datos de prueba:

Campo	Valor de prueba
Nombre completo	Estudiante Prueba
Email	estudiante@test.com (usa un email real para recibir la confirmación)
Contraseña	123456
Rol	Estudiante

Carrera	Ingeniería de Sistemas
---------	------------------------

- Haz clic en **Crear Cuenta**. Deberías ver el mensaje de verificación.
- Revisa tu email y haz clic en el enlace de confirmación.
- Ve a `http://localhost:3000/login` e ingresa las credenciales.
- Deberías ser redirigido a `/estudiante` (la página mostrará un 404, es normal — la crearemos en la Entrega 3).

Tip: Deshabilitar confirmación para pruebas

Para agilizar las pruebas, puedes ir a Supabase > Authentication > Providers > Email y desactivar temporalmente "Confirm email". Recuerda activarlo de nuevo antes de producción.

- Verifica en Supabase Dashboard:
 - Authentication > Users:** deberías ver el nuevo usuario.
 - Table Editor > usuarios_perfil:** deberías ver el perfil con nombre, rol y carrera.

Verificación Final

Confirma todos estos puntos antes de avanzar a la Entrega 3A:

- ✓ `app/(auth)/layout.tsx` centra el contenido en la pantalla
- ✓ `http://localhost:3000/login` muestra el formulario de login
- ✓ `http://localhost:3000/registro` muestra el formulario de registro
- ✓ Registrar un usuario crea el registro en `auth.users` Y en `usuarios_perfil`
- ✓ Login redirige correctamente: estudiante → `/estudiante`, docente → `/docente`, admin → `/admin`
- ✓ Si visitas `/login` estando autenticado, te redirige a tu dashboard
- ✓ Los enlaces "Regístrate aquí" e "Inicia sesión" navegan correctamente entre páginas

Ejercicio Propuesto

- Crea tres usuarios de prueba:** Registra un estudiante, un docente y un admin. Verifica que cada uno es redirigido a su dashboard correcto después del login.
- Manejo de errores:** Intenta registrar dos veces con el mismo email. ¿Qué mensaje de error ves? ¿Es claro para el usuario?

11. **Investiga el token:** Después de hacer login, abre DevTools > Application > Cookies.
¿Cuántas cookies creó Supabase? ¿Qué nombres tienen?
 12. **Pregunta de reflexión:** ¿Por qué la API Route de registro usa la Service Role Key en lugar de la clave pública? ¿Qué pasaría si un usuario malicioso enviara rol: "admin" en el body?
-

Próximo: Entrega 3A

En la siguiente entrega crearemos el layout del dashboard con el Sidebar de shadcn/ui, navegación dinámica por rol, y el layout raíz con fuentes y providers. Es donde la app empieza a verse como una aplicación real.